

Jems AI Framework (JAF) - Architecture Guide

Version: 1.0

Author: Alex (Team Lead / Senior Architect)

Date: 30 Nov 2024

Status:  Approved for Implementation

Purpose: This document serves as the authoritative architecture guide for all developers implementing the JAF framework over the 4-week sprint.

Table of Contents

1. [Vision & Principles](#)
 2. [System Architecture](#)
 3. [Core Components](#)
 4. [The Three Execution Modes](#)
 5. [Data Models](#)
 6. [Process Lifecycle](#)
 7. [Implementation Patterns](#)
 8. [API Contracts](#)
 9. [Persistence Strategy](#)
 10. [Event-Driven Architecture](#)
 11. [Error Handling & Recovery](#)
 12. [Testing Strategy](#)
 13. [Deployment Architecture](#)
-

1. Vision & Principles

1.1 What is JAF?

Jems AI Framework (JAF) is an **enterprise-grade Actor-based orchestration framework** that enables three types of automation workflows from a single runtime:

1. **Agentic Flow:** Single orchestrator + multiple deterministic tools
2. **Multi-Agent Flow:** Orchestrator delegates to specialized agent actors
3. **Traditional Automation:** Deterministic workflows without LLM (BPMN-like)

1.2 Core Design Principles

1.2.1 Actor Model (Share Nothing Architecture)

- Actors never share state directly
- Communication exclusively via asynchronous messages
- Enables: location transparency, fault tolerance, scalability
- **Scientific Basis:** Carl Hewitt (1973), proven by Erlang/Akka systems

1.2.2 Engine Abstraction (Vendor Independence)

- Actors never call execution frameworks directly (no CrewAI imports in actor code)
- Engine interface abstracts planning and execution
- Can swap backends: CrewAI → Native → LangGraph → Vertex AI
- **Goal:** Eliminate vendor lock-in

1.2.3 Hybrid Intelligence (Cost Optimization)

- **Cheap LLM** (gpt-4o-mini): Tool routing, formatting (~\$0.15/1M tokens)
- **Expensive LLM** (gpt-4o): Planning, complex reasoning (~\$5/1M tokens)
- **No LLM:** Deterministic flows (compliance-critical tasks)
- **Goal:** 50% cost reduction vs current system

1.2.4 Persistence-First (Enterprise Reliability)

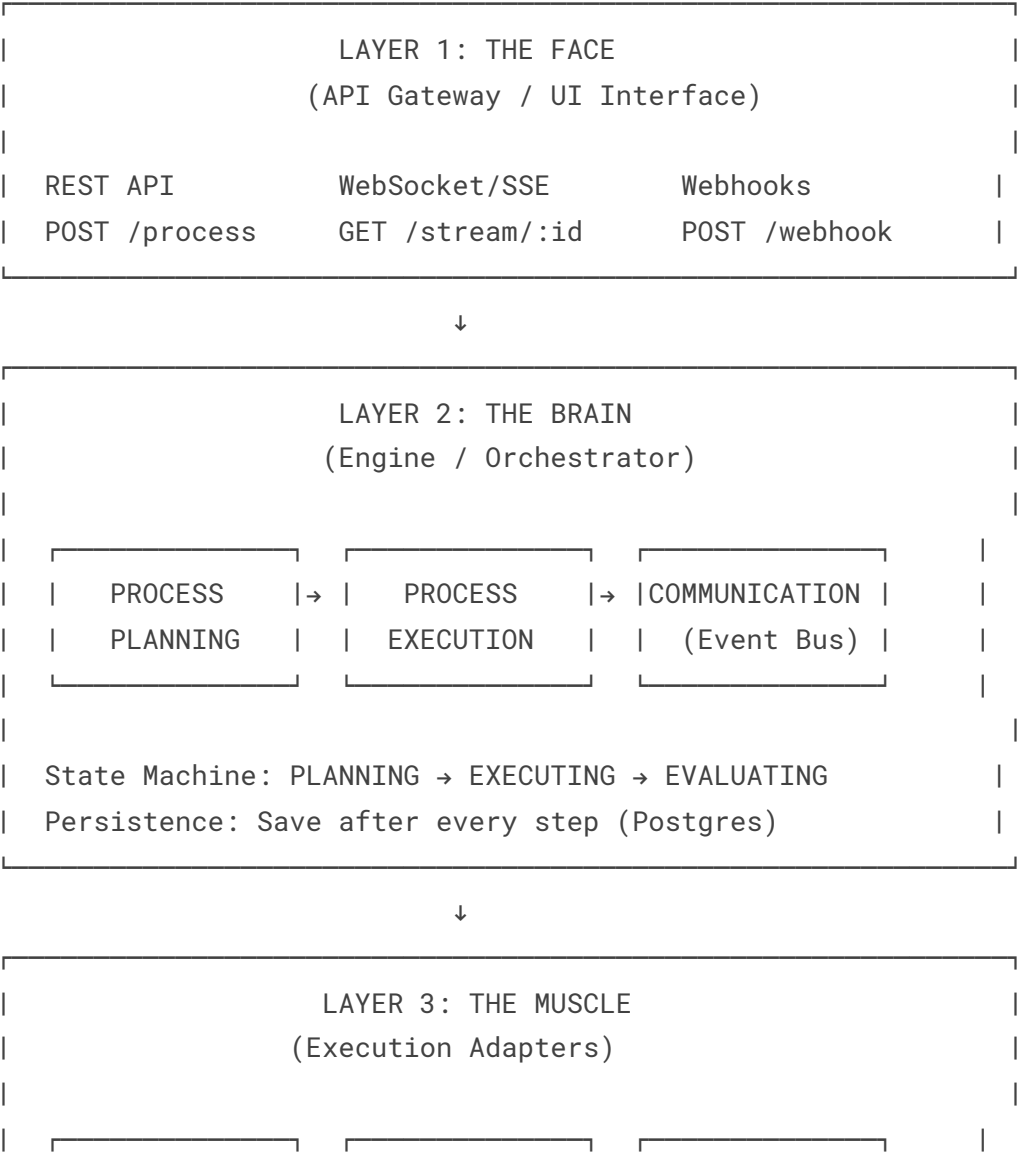
- Every state transition saved to Postgres
- Process can resume exactly where it crashed
- Enables: human-in-the-loop, long-running workflows (days/weeks), audit trails
- **Goal:** Zero data loss, 99.9% reliability

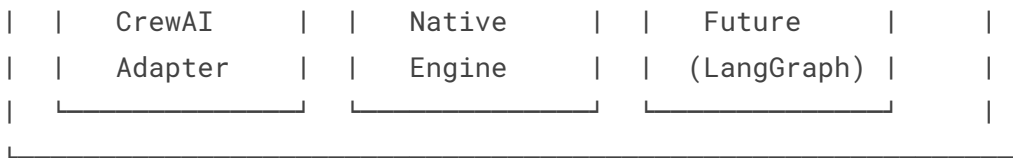
1.2.5 Event-Driven Observability (Real-Time Monitoring)

- Engine emits events on every step
- Frontend receives via SSE (Server-Sent Events)
- Events match strict contract: AgentAction type
- **Goal:** Real-time visibility into process execution

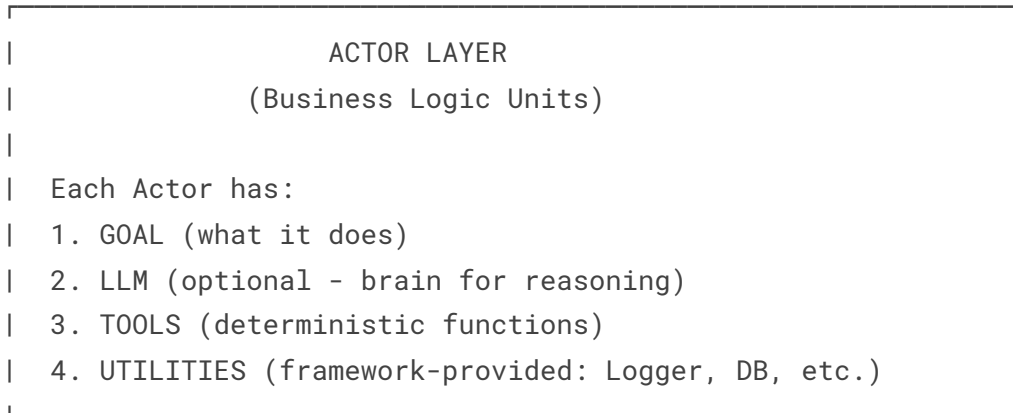
2. System Architecture

2.1 High-Level Architecture (The Three Layers)



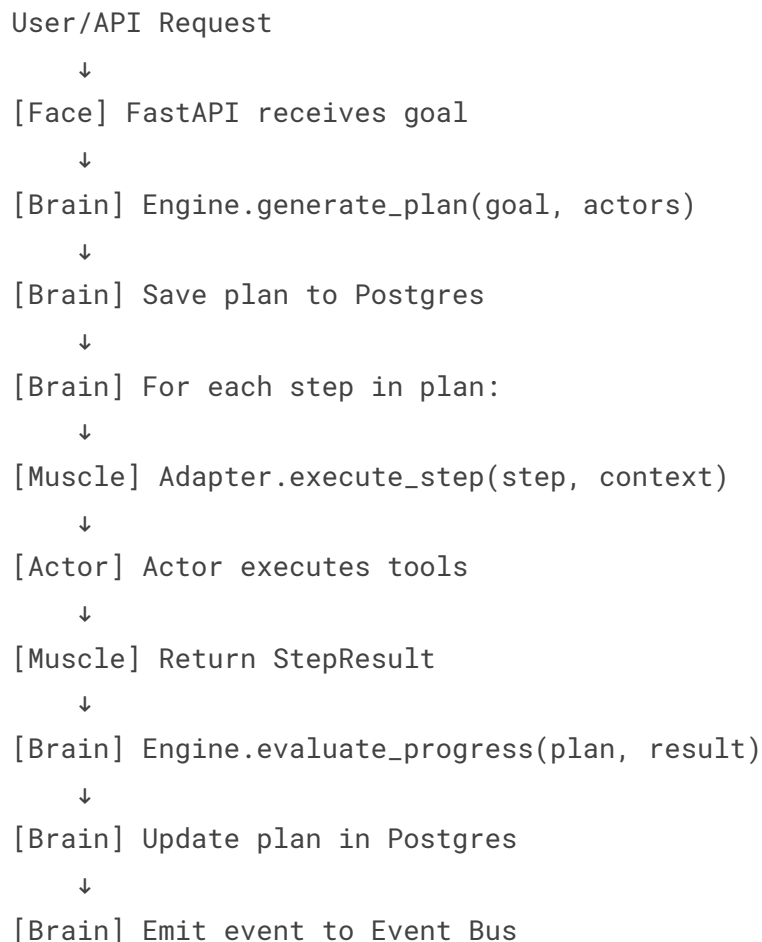


↓



2.2 Component Interaction Flow

Request Flow (Synchronous):



↓
[Face] SSE streams event to frontend
↓
Repeat until plan complete
↓
[Face] Return final result to user

Event Flow (Asynchronous):

Engine emits AgentAction
↓
Event Bus broadcasts
↓
Subscribers receive:
- Console Logger (prints to terminal)
- SSE Endpoint (streams to frontend)
- Persistence (saves to events table)
- Metrics (updates Prometheus)

3. Core Components

3.1 The Actor (System Object)

Definition: The atomic unit of functionality. An Actor is a **stateless definition** (like a recipe) that can be instantiated into **stateful processes** (like meals).

Analogy:

- **Actor** = Recipe (reusable, shareable)
- **Process** = Meal (one-time instance)

Properties:

```
class Actor(ABC):  
    name: str                # Unique identifier (e.g., "ProvisionningAct  
    description: str          # Human-readable purpose  
    goal: str                 # What this actor achieves
```

```

tools: List[Callable]           # Deterministic functions (business logic)
utilities: List[Callable]       # Framework-provided (Logger, DB, etc.)
llm_config: Optional[LLMConfig] # Optional: None = deterministic, set = prob

@abstractmethod
def configure(self):
    """Setup tools, LLM, utilities. Called once on initialization."""
    pass

```

File Location: framework/actor/base_actor.py (JAF-14, JAF-15)

Hybrid Modes:

- **Deterministic Mode:** llm_config = None → Pure Python logic (if/else)
- **Probabilistic Mode:** llm_config = LLMConfig(model="gpt-4o-mini") → LLM makes decisions

Example - Deterministic Actor:

```

class ValidationActor(Actor):
    """Validates inputs without LLM (compliance-critical)."""

    def configure(self):
        self.name = "ValidationActor"
        self.goal = "Validate employee data against rules"
        self.tools = [self.validate_email, self.validate_cuid]
        self.llm_config = None # No AI, pure logic

    def validate_email(self, email: str) -> bool:
        return "@orange.com" in email # Hardcoded rule

```

Example - Probabilistic Actor:

```

class ProvisioningActor(Actor):
    """Provisions hardware with LLM reasoning."""

    def configure(self):
        self.name = "ProvisioningActor"
        self.goal = "Provision PC and phone for employees"
        self.tools = [

```

```

        self.check_eligibility,
        self.select_model,
        self.order_product,
        # ... 7 tools total
    ]
    self.llm_config = LLMConfig(model="gpt-4o-mini", temperature=0.0)

def check_eligibility(self, cuid: str) -> Dict:
    # Calls external API, returns structured data
    pass

```

3.2 The Engine (Dynamic Planner)

Definition: The “brain” that manages process lifecycle. The Engine is **abstract** - implementations (adapters) handle execution.

Interface:

```

class EngineAbstraction(ABC):
    """The heart of JAF - manages process lifecycle."""

    @abstractmethod
    async def generate_plan(self, goal: str, available_actors: List[Actor]) -> Plan:
        """Phase 1: PROCESS PLANNING

        Input: Natural language goal + list of available actors
        Output: Plan object with ordered steps

        Example:
        Goal: "Onboard Alice with PC"
        Output: Plan([
            PlanStep(actor="PiramidActor", tool="classify_identity"),
            PlanStep(actor="ProvisionningActor", tool="provision_pc"),
        ])
        """
        pass

    @abstractmethod
    async def execute_step(self, step: PlanStep, context: dict) -> StepResult:

```

```
"""Phase 2: PROCESS EXECUTION
```

```
Input: Single step from plan + shared context
```

```
Output: StepResult with output/error
```

```
Implementation:
```

1. Get Actor from registry
2. Create runtime agent/process
3. Execute tool with parameters
4. Return structured result

```
"""
```

```
pass
```

```
@abstractmethod
```

```
async def evaluate_progress(self, plan: Plan, result: StepResult) -> PlanUpdate
```

```
"""Phase 3: EVALUATION (Re-Think)
```

```
Input: Current plan + latest step result
```

```
Output: PlanUpdate (continue/modify plan/abort)
```

```
Decision Tree:
```

- If result.status == "success" → Continue to next step
- If result.status == "error" → Retry or modify plan
- If goal achieved → Mark plan as COMPLETED

```
"""
```

File Location: framework/engine/engine_abstraction.py (JAF-2, JAF-3)

Implementations:

- **CrewAIAdapter** (framework/engine/crewai_adapter.py) - JAF-5 (Week 2)
 - Wraps existing CrewAI logic
 - Backward compatible with current system
 - Used for: Complex multi-agent flows, delegation
- **NativeEngine** (framework/engine/native_engine.py) - JAF-15 (Week 3, optional)
 - Lightweight, uses OpenAI function calling directly
 - 5x faster than CrewAI for simple workflows
 - Used for: High-volume, deterministic tasks

Engine Selection (Auto):

```
# framework/engine/factory.py (JAF-16)
class EngineFactory:
    @staticmethod
    def create(actor: Actor, mode: str = "auto") -> EngineAbstraction:
        """Auto-select best engine based on actor config."""

        if mode == "auto":
            # Use Native for simple actors, CrewAI for complex
            if actor.llm_config is None:
                return NativeEngine() # Deterministic, no LLM needed
            elif len(actor.tools) < 5:
                return NativeEngine() # Simple, Native is faster
            else:
                return CrewAIAdapter() # Complex, use CrewAI delegation

        elif mode == "crewai":
            return CrewAIAdapter()
        elif mode == "native":
            return NativeEngine()
```

3.3 The Process (Runtime Instance)

Definition: A **stateful instance** of a workflow execution. One Actor (recipe) can spawn thousands of Processes (meals).

Properties:

```
class Process:
    id: UUID # Unique process identifier
    goal: str # Original goal/request
    status: ProcessStatus # PLANNING, EXECUTING, WAITING_INPUT, COMPLETED, FA
    current_plan: Plan # The plan (list of steps)
    context: Dict[str, Any] # Shared memory between steps
    created_at: datetime # Process start time
    updated_at: datetime # Last state change
```

Database Schema:

```
CREATE TABLE processes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  goal TEXT NOT NULL,  
  status process_status NOT NULL DEFAULT 'PLANNING',  
  current_plan JSONB NOT NULL,  
  context JSONB DEFAULT '{}',  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

File Location: framework/persistence/models.py (JAF-23, JAF-24)

Lifecycle:

1. User submits goal
 2. Process created (status = PLANNING)
 3. Engine generates plan
 4. Status → EXECUTING
 5. For each step:
 - Execute step
 - Save result to context
 - Update process state in DB
 - Emit event
 6. Status → COMPLETED (or FAILED)
 7. Return final result
-

3.4 The Actor Registry

Definition: A singleton that discovers, registers, and retrieves Actors at runtime.

Purpose:

- Dynamic actor discovery (no hardcoded imports)
- Backward compatible with `agents.yaml`
- Enables “plug-and-play” actor deployment

Interface:

```

class ActorRegistry:
    """Singleton registry for all actors."""

    _instance = None
    _actors: Dict[str, Type[Actor]] = {}

    @classmethod
    def register(cls, name: str, actor_class: Type[Actor]):
        """Register an actor by name."""
        cls._actors[name] = actor_class

    @classmethod
    def get(cls, name: str) -> Actor:
        """Retrieve actor instance by name."""
        actor_class = cls._actors.get(name)
        if not actor_class:
            raise ValueError(f"Actor '{name}' not found in registry")

        instance = actor_class()
        instance.configure() # Setup tools, LLM
        return instance

    @classmethod
    def list(cls) -> List[str]:
        """List all registered actor names."""
        return list(cls._actors.keys())

    @classmethod
    def discover_from_yaml(cls, yaml_path: str = "agents.yaml"):
        """Auto-discover actors from YAML file (backward compat)."""
        # Parse agents.yaml, register each actor
        pass

```

File Location: framework/actor/registry.py (JAF-17, JAF-18, JAF-19)

Usage:

```

# At startup
ActorRegistry.discover_from_yaml("agents.yaml")

# In engine
actor = ActorRegistry.get("ProvisionningActor")

```

4. The Three Execution Modes

4.1 Mode 1: Agentic Flow (Single Actor + Tools)

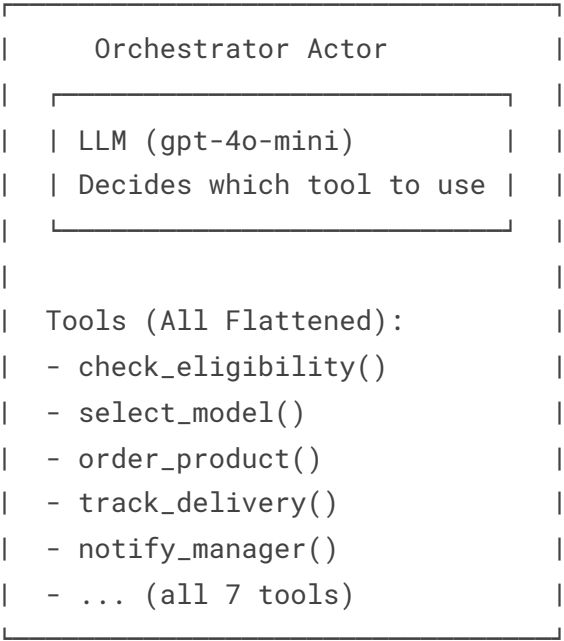
Use Case: High-volume, simple tasks requiring minimal reasoning

Cost: ~\$0.05 per workflow

Latency: ~2 seconds

LLM Usage: Single orchestrator with cheap LLM (gpt-4o-mini)

Architecture:



When to Use:

- Simple workflows (3-5 steps)
- All tools belong to one domain (e.g., provisioning only)
- Cost is priority
- No specialized expertise needed

Example Workflow:

```
# Agentic Flow: Provision PC
Plan([
    PlanStep(actor="ProvisionningActor", tool="check_eligibility", params={"cuid":
    PlanStep(actor="ProvisionningActor", tool="select_model", params={"cuid": "C123
    PlanStep(actor="ProvisionningActor", tool="order_product", params={"model": "De
])
```



Implementation: JAF-9, JAF-10, JAF-32, JAF-33

4.2 Mode 2: Multi-Agent Flow (Orchestrator + Specialized Agents)

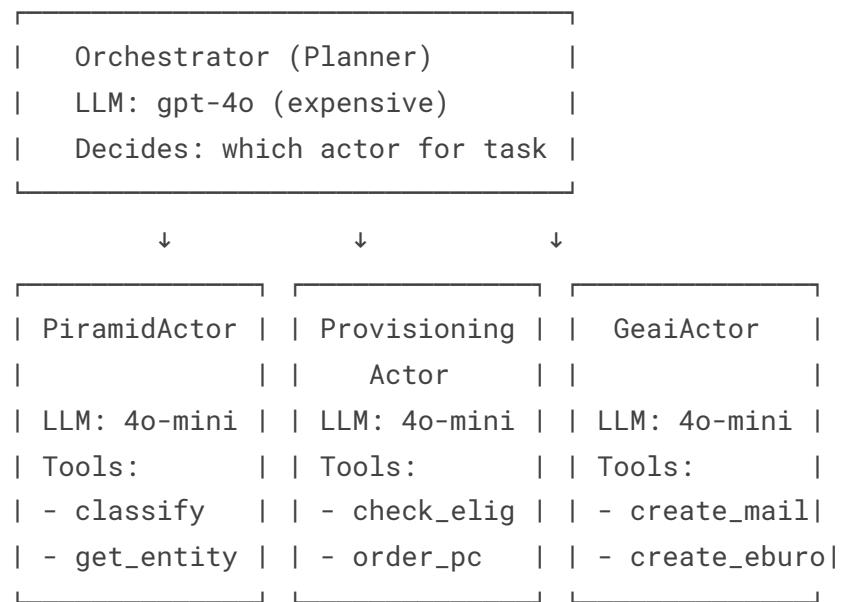
Use Case: Complex workflows requiring specialized expertise and reasoning

Cost: ~\$0.12 per workflow

Latency: ~5 seconds

LLM Usage: Multiple agents, each with own LLM context

Architecture:



When to Use:

- Complex workflows (5+ steps)
- Multiple domains (identity + provisioning + access control)

- Specialized expertise required
- Nested processes (actor calls sub-actor)

Example Workflow:

```
# Multi-Agent Flow: Full Onboarding
Plan([
    PlanStep(actor="PiramidActor", tool="classify_identity", params={"cuid": "C123"}),
    PlanStep(actor="ProvisionningActor", tool="provision_pc", params={"cuid": "C123"}),
    PlanStep(actor="GeaiActor", tool="create_email", params={"cuid": "C123"}),
    PlanStep(actor="HabilitationActor", tool="grant_permissions", params={"cuid": "C123"}),
])
```

Key Difference from Agentic:

- **Agentic:** 1 LLM call, all tools in one actor
- **Multi-Agent:** 4 LLM calls, each actor has specialized context/knowledge

Implementation: JAF-11, JAF-12, JAF-34, JAF-35

4.3 Mode 3: Traditional Automation (No LLM)

Use Case: Deterministic, compliance-critical tasks

Cost: \$0.00 per workflow (no LLM calls)

Latency: ~0.5 seconds

LLM Usage: None - pure if/else logic

Architecture:

```

| Deterministic Actor |
| (llm_config = None) |
|                       |
| Hardcoded Decision Tree: |
| if user.role == "Developer": |
|     return "Offer A" |
| elif user.role == "Manager": |

```

```
|         return "Offer B"         |
|     else:                         |
|         return "No equipment"    |
|_____|
```

When to Use:

- Regulatory/compliance workflows (audit trail required)
- 100% deterministic (no variation allowed)
- Cost is critical (high volume)
- BPMN/Camunda replacement

Example Workflow:

```
# Traditional Automation: Standard Provisioning
class StandardProvisioningActor(Actor):
    def configure(self):
        self.llm_config = None # No LLM

    def execute(self, user: dict) -> dict:
        # Hardcoded logic (no LLM calls)
        if user["role"] == "Developer":
            model = "Dell Latitude 7420"
        elif user["role"] == "Manager":
            model = "Dell Latitude 5420"
        else:
            raise ValueError("Invalid role")

        return self.order_product(user["cuid"], model)
```

Implementation: JAF-38, JAF-39, JAF-40 (Nice-to-have, Week 4 optional)

5. Data Models

5.1 Core Types (framework/types.py)

LLMConfig:

```

from pydantic import BaseModel, Field
from typing import Optional

class LLMConfig(BaseModel):
    """LLM configuration for Actor."""
    model: str = Field(..., description="Model name (e.g., gpt-4o, gpt-4o-mini)")
    temperature: float = Field(0.0, ge=0.0, le=2.0)
    max_tokens: Optional[int] = Field(None, gt=0)

    class Config:
        frozen = True # Immutable

```

PlanStep:

```

class PlanStep(BaseModel):
    """A single step in an execution plan."""
    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    actor_name: str # Which actor executes this (e.g., "ProvisioningActor")
    tool_name: str # Which tool to call (e.g., "check_eligibility")
    parameters: Dict[str, Any] # Tool input parameters
    status: Literal["pending", "executing", "completed", "failed"] = "pending"

```

Plan:

```

class Plan(BaseModel):
    """A complete execution plan."""
    steps: List[PlanStep]
    status: Literal["PLANNING", "EXECUTING", "COMPLETED", "FAILED"] = "PLANNING"
    created_at: datetime = Field(default_factory=datetime.utcnow)

    def next_step(self) -> Optional[PlanStep]:
        """Get next pending step."""
        for step in self.steps:
            if step.status == "pending":
                return step
        return None

```

StepResult:

```

class StepResult(BaseModel):
    """Result of executing a single step."""

```

```

output: Dict[str, Any] # Tool return value
status: Literal["success", "error"]
error: Optional[str] = None # Error message if status == "error"
execution_time: float = 0.0 # Seconds
cost: float = 0.0 # USD (LLM token cost)

```

PlanUpdate:

```

class PlanUpdate(BaseModel):
    """Engine's decision after evaluating a step result."""
    action: Literal["continue", "modify", "retry", "abort"]
    modified_plan: Optional[Plan] = None # New plan if action == "modify"
    reason: str # Why this decision was made

```

AgentAction (Event):

```

class AgentAction(BaseModel):
    """Event emitted during process execution.

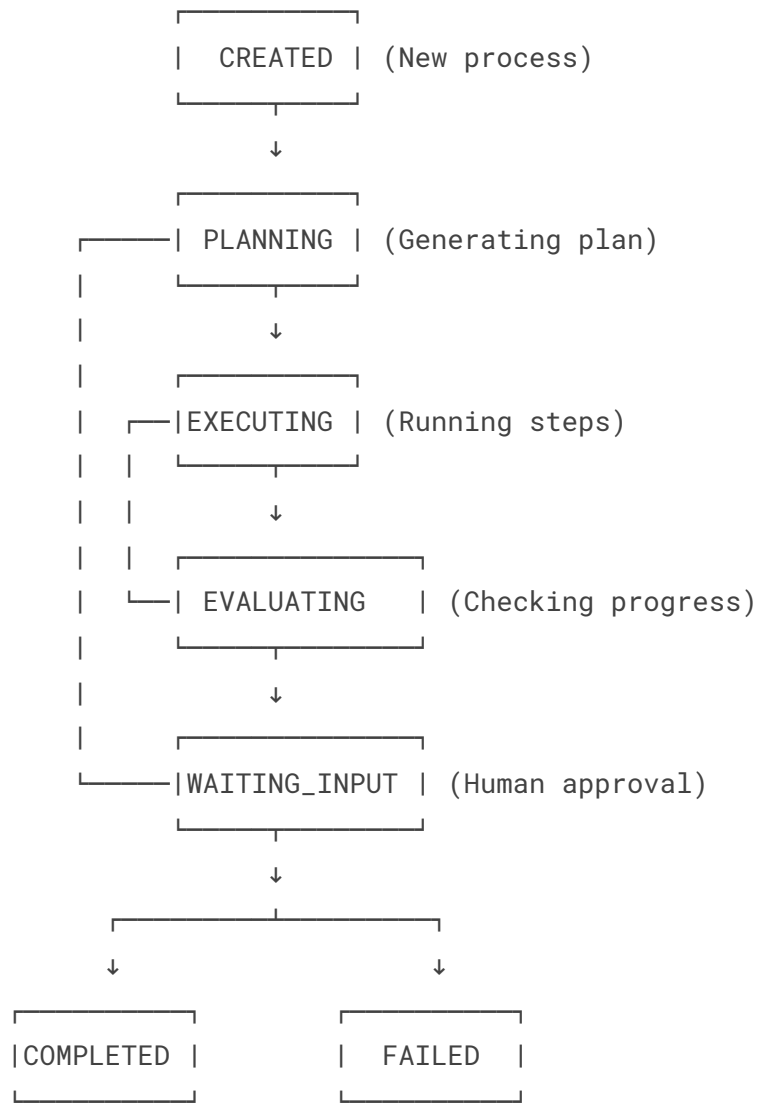
    Matches TypeScript contract in frontend:
    type AgentAction = {
      agentId: string;
      actionType: string;
      message: string;
      status: "complete" | "error";
      timestamp: string;
    };
    """
    agentId: str # Actor name (e.g., "ProvisioningActor")
    actionType: str # Tool name (e.g., "check_eligibility") or "thought" or "plan"
    message: str # Human-readable description (e.g., "Checking eligibility for C12")
    status: Literal["complete", "error"]
    timestamp: str = Field(default_factory=lambda: datetime.utcnow().isoformat() +

```

File Location: framework/types.py (JAF-4, JAF-16, JAF-29)

6. Process Lifecycle

6.1 State Machine



State Transitions:

```
# framework/state_machine.py (JAF-19)
class ProcessStateMachine:
    """Manages process state transitions."""

    VALID_TRANSITIONS = {
        "CREATED": ["PLANNING"],
        "PLANNING": ["EXECUTING", "FAILED"],
        "EXECUTING": ["EVALUATING", "FAILED"],
        "EVALUATING": ["EXECUTING", "WAITING_INPUT", "COMPLETED", "FAILED"],
        "WAITING_INPUT": ["EXECUTING", "FAILED"],
        "COMPLETED": [], # Terminal state
```

```

        "FAILED": [],      # Terminal state
    }

    async def transition(self, process_id: UUID, new_status: str):
        """Transition process to new status with validation."""
        current = await self.repo.get_status(process_id)

        if new_status not in self.VALID_TRANSITIONS[current]:
            raise InvalidTransitionError(f"Cannot go from {current} to {new_status}")

        await self.repo.update_status(process_id, new_status)
        await self.event_bus.emit(AgentAction(
            agentId="StateMachine",
            actionType="state_change",
            message=f"Process {process_id} → {new_status}",
            status="complete"
        ))

```

File Location: framework/state_machine.py (JAF-19, Week 3)

6.2 The Plan-Execute-Evaluate Loop

Full Lifecycle:

```

# framework/orchestrator.py (Future - Week 3+)
class Orchestrator:
    """Main orchestration loop."""

    async def run(self, goal: str) -> Dict[str, Any]:
        """Execute a goal end-to-end."""

        # 1. CREATE PROCESS
        process_id = uuid.uuid4()
        await self.repo.create(process_id, goal, status="CREATED")

        # 2. PLANNING PHASE
        await self.state_machine.transition(process_id, "PLANNING")
        actors = ActorRegistry.list()
        plan = await self.engine.generate_plan(goal, actors)
        await self.repo.update_plan(process_id, plan)

```

```
# 3. EXECUTION PHASE
```

```
await self.state_machine.transition(process_id, "EXECUTING")
context = {}
```

```
while True:
```

```
    step = plan.next_step()
```

```
    if not step:
```

```
        break # All steps complete
```

```
    # Execute step
```

```
    result = await self.engine.execute_step(step, context)
```

```
    # Save result to context
```

```
    context[step.id] = result.output
```

```
    # Persist state
```

```
    await self.repo.update_context(process_id, context)
```

```
    # Emit event
```

```
    await self.event_bus.emit(AgentAction(
        agentId=step.actor_name,
        actionType=step.tool_name,
        message=f"Completed {step.tool_name}",
        status="complete" if result.status == "success" else "error"
    ))
```

```
# 4. EVALUATION PHASE
```

```
await self.state_machine.transition(process_id, "EVALUATING")
```

```
update = await self.engine.evaluate_progress(plan, result)
```

```
if update.action == "abort":
```

```
    await self.state_machine.transition(process_id, "FAILED")
```

```
    return {"error": update.reason}
```

```
elif update.action == "modify":
```

```
    plan = update.modified_plan
```

```
    await self.repo.update_plan(process_id, plan)
```

```
# Go back to EXECUTING
```

```
await self.state_machine.transition(process_id, "EXECUTING")
```

```
# 5. COMPLETION
```

```
await self.state_machine.transition(process_id, "COMPLETED")
return {"result": context, "process_id": str(process_id)}
```

7. Implementation Patterns

7.1 Creating a New Actor

Step 1: Define Actor Class

```
# framework/actors/my_new_actor.py
from framework.actor.base_actor import Actor
from framework.types import LLMConfig
from typing import Dict

class MyNewActor(Actor):
    """Description of what this actor does."""

    def configure(self):
        """Setup tools and LLM config."""
        self.name = "MyNewActor"
        self.description = "Short description for orchestrator"
        self.goal = "Specific goal this actor achieves"

        # Register tools (methods of this class)
        self.tools = [
            self.my_tool_1,
            self.my_tool_2,
        ]

        # Optional: Set LLM config (None = deterministic)
        self.llm_config = LLMConfig(model="gpt-4o-mini", temperature=0.0)

    def my_tool_1(self, param1: str, param2: int) -> Dict:
        """Tool description (used by LLM to decide when to call)."""
        # Business logic here
        return {"result": "success", "data": {...}}

    def my_tool_2(self, param: str) -> Dict:
```

```
"""Another tool."""  
return {"output": "..."}  

```

Step 2: Register Actor

```
# At startup or in agents/__init__.py  
from framework.actor.registry import ActorRegistry  
from framework.actors.my_new_actor import MyNewActor  

```

```
ActorRegistry.register("MyNewActor", MyNewActor)
```

Step 3: Use in Workflow

```
# The actor is now available to the Engine  
actors = ActorRegistry.list() # Includes "MyNewActor"  
plan = await engine.generate_plan("Do something", actors)
```

7.2 Implementing an Engine Adapter

Step 1: Inherit from EngineAbstraction

```
# framework/engine/my_custom_adapter.py  
from framework.engine.engine_abstraction import EngineAbstraction  
from framework.types import Plan, PlanStep, StepResult, PlanUpdate  
  
class MyCustomAdapter(EngineAbstraction):  
    """Custom engine implementation."""  
  
    async def generate_plan(self, goal: str, available_actors: List[Actor]) -> Plan:  
        """Implement planning logic."""  
        # Option 1: Use LLM to generate plan  
        # Option 2: Use hardcoded rules  
        # Option 3: Hybrid (rules + LLM)  
  
        steps = []  
        # ... generate steps ...  
  
        return Plan(steps=steps, status="PLANNING")
```

```

async def execute_step(self, step: PlanStep, context: dict) -> StepResult:
    """Implement execution logic."""
    # 1. Get actor
    actor = ActorRegistry.get(step.actor_name)

    # 2. Get tool
    tool = getattr(actor, step.tool_name)

    # 3. Execute
    try:
        output = tool(**step.parameters)
        return StepResult(output=output, status="success")
    except Exception as e:
        return StepResult(output={}, status="error", error=str(e))

async def evaluate_progress(self, plan: Plan, result: StepResult) -> PlanUpdate:
    """Implement evaluation logic."""
    if result.status == "success":
        return PlanUpdate(action="continue", reason="Step succeeded")
    else:

```

Step 2: Register in Factory

```

# framework/engine/factory.py
from framework.engine.my_custom_adapter import MyCustomAdapter

class EngineFactory:
    ENGINES = {
        "crewai": CrewAIAdapter,
        "native": NativeEngine,
        "custom": MyCustomAdapter, # Add new engine
    }

```

7.3 Migrating Existing Agents to Actors

Before (Current System):

```
# agents/provisionnement_agent.py (OLD)
class ProvisionningAgent:
    def __init__(self, base_url: str):
        self.name = "ProvisionnementAgent"
        self.base_url = base_url

    def check_eligibility(self, cuid: str) -> Dict:
        # ... implementation ...
        pass
```

After (New Framework):

```
# agents/provisionnement_agent.py (NEW)
from framework.actor.base_actor import Actor
from framework.types import LLMConfig

class ProvisionningAgent(Actor):
    """Hardware provisioning actor - migrated to JAF framework."""

    def __init__(self, base_url: str = "http://localhost:8089"):
        self.base_url = base_url # Keep existing config
        super().__init__() # Initialize Actor base class

    def configure(self):
        """Setup tools and LLM - NEW METHOD."""
        self.name = "ProvisionnementActor"
        self.description = "Provide hardware equipment to employees"
        self.goal = "Provision PC and phone for new/existing employees"

        # Register all tools
        self.tools = [
            self.check_eligibility,
            # ... other tools
        ]

        # Set LLM config
        self.llm_config = LLMConfig(model="gpt-4o-mini", temperature=0.0)

# ALL EXISTING METHODS STAY UNCHANGED
def check_eligibility(self, cuid: str) -> Dict:
    # ... same implementation, no changes ...
    pass
```

Migration Checklist:

1. ☒ Import Actor base class
2. ☒ Inherit: `class MyAgent(Actor)`
3. ☒ Add `configure()` method
4. ☒ Move tool registration to `configure()`
5. ☒ Set `llm_config` (or `None` for deterministic)
6. ☒ Keep all existing methods unchanged
7. ☒ Run regression tests (must pass)

Implementation: JAF-20, JAF-21 (Week 1-2)

8. API Contracts

8.1 Frontend Contract (AgentAction Event)

TypeScript Definition (Frontend):

```
// frontend/types/agent-action.ts
type AgentAction = {
  agentId: string;           // Actor name (e.g., "ProvisionningActor")
  actionType: string;        // Tool name (e.g., "check_eligibility") or "thought"
  message: string;           // Human-readable (e.g., "Checking eligibility for C12345")
  status: "complete" | "error";
  timestamp: string;         // ISO 8601 format
};
```

Python Definition (Backend):

```
# framework/types.py
class AgentAction(BaseModel):
    agentId: str
    actionType: str
    message: str
```

```
status: Literal["complete", "error"]
```

⚠️ **CRITICAL:** These must match EXACTLY. Any mismatch breaks frontend integration.

Implementation: JAF-29 (Week 1)

8.2 REST API Endpoints

Process Execution:

```
POST /api/processes
Content-Type: application/json
```

Request:

```
{
  "goal": "Onboard employee Alice (cuid: C123456) with PC",
  "mode": "multi-agent", // Optional: "agentic" | "multi-agent" | "traditional"
  "context": {           // Optional: Initial context
    "cuid": "C123456",
    "manager_cuid": "C789012"
  }
}
```

Response:

```
{
  "process_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "PLANNING",
  "created_at": "2024-11-30T10:00:00Z"
}
```

Get Process Status:

```
GET /api/processes/{process_id}
```

Response:

```
{
  "process_id": "550e8400-...",
  "status": "EXECUTING",
}
```

```

"current_plan": {
  "steps": [
    {"id": "step-1", "actor": "PyramidActor", "status": "completed"},
    {"id": "step-2", "actor": "ProvisionningActor", "status": "executing"}
  ]
},
"context": {
  "identity": "Internal/DSI",
  "model": "Dell Latitude 7420"
},
"progress": "50%"
}

```

Stream Events (SSE):

GET /api/processes/{process_id}/stream

Accept: text/event-stream

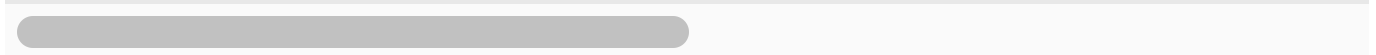
Response (Server-Sent Events):

data: {"agentId":"PyramidActor","actionType":"classify_identity","message":"Classif

data: {"agentId":"ProvisionningActor","actionType":"check_eligibility","message":"C

data: {"agentId":"ProvisionningActor","actionType":"order_product","message":"Order

data: {"process_id":"550e8400-...", "status":"COMPLETED", "timestamp":"2024-11-30T10:



Implementation: JAF-14, JAF-15 (Week 2-3)

9. Persistence Strategy

9.1 Database Schema

Tables:

```

-- Processes Table
CREATE TYPE process_status AS ENUM ('PLANNING', 'EXECUTING', 'EVALUATING', 'WAITING');

CREATE TABLE processes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    goal TEXT NOT NULL,
    status process_status NOT NULL DEFAULT 'PLANNING',
    current_plan JSONB NOT NULL,
    context JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_processes_status ON processes(status);
CREATE INDEX idx_processes_updated_at ON processes(updated_at DESC);

-- Events Table (Optional - for audit trail)
CREATE TYPE event_type AS ENUM ('TOOL_CALL', 'THOUGHT', 'ERROR', 'STATE_CHANGE');

CREATE TABLE events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    process_id UUID REFERENCES processes(id) ON DELETE CASCADE,
    type event_type NOT NULL,
    payload JSONB NOT NULL,
    timestamp TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_events_process_id ON events(process_id);
CREATE INDEX idx_events_timestamp ON events(timestamp DESC);

```

Implementation: JAF-24 (Week 1)

9.2 Repository Pattern

Interface:

```

# framework/persistence/process_repo.py (JAF-25)
from sqlalchemy.ext.asyncio import AsyncSession

```

```

from framework.types import Plan

class ProcessRepository:
    """Async repository for process CRUD operations."""

    def __init__(self, session: AsyncSession):
        self.session = session

    async def create(self, process_id: UUID, goal: str, status: str = "PLANNING") -
        """Create new process."""
        query = """
            INSERT INTO processes (id, goal, status, current_plan, context)
            VALUES (:id, :goal, :status, :plan, :context)
        """
        await self.session.execute(query, {
            "id": process_id,
            "goal": goal,
            "status": status,
            "plan": json.dumps({"steps": []}),
            "context": json.dumps({})
        })
        await self.session.commit()

    async def get(self, process_id: UUID) -> Optional[Dict]:
        """Get process by ID."""
        query = "SELECT * FROM processes WHERE id = :id"
        result = await self.session.execute(query, {"id": process_id})
        return result.fetchone()

    async def update_status(self, process_id: UUID, status: str) -> None:
        """Update process status."""
        query = """
            UPDATE processes
            SET status = :status, updated_at = NOW()
            WHERE id = :id
        """
        await self.session.execute(query, {"id": process_id, "status": status})
        await self.session.commit()

    async def update_plan(self, process_id: UUID, plan: Plan) -> None:
        """Update current plan."""
        query = """
            UPDATE processes

```

```

        SET current_plan = :plan, updated_at = NOW()
        WHERE id = :id
    """
    await self.session.execute(query, {
        "id": process_id,
        "plan": plan.json()
    })
    await self.session.commit()

async def update_context(self, process_id: UUID, context: Dict) -> None:
    """Update shared context."""
    query = """
        UPDATE processes
        SET context = :context, updated_at = NOW()
        WHERE id = :id
    """
    await self.session.execute(query, {
        "id": process_id,
        "context": json.dumps(context)
    })
    await self.session.commit()

async def log_event(self, process_id: UUID, event_type: str, payload: Dict) ->
    """Log event to events table."""
    query = """
        INSERT INTO events (process_id, type, payload)
        VALUES (:process_id, :type, :payload)
    """
    await self.session.execute(query, {
        "process_id": process_id,
        "type": event_type,
        "payload": json.dumps(payload)
    })

```

Connection Management:

```

# framework/persistence/database.py
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "postgresql+asyncpg://user:pass@localhost/jaf"

```

```

engine = create_async_engine(DATABASE_URL, echo=True, pool_size=10)
async_session_maker = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def get_session() -> AsyncSession:
    """Dependency injection for FastAPI."""
    async with async_session_maker() as session:

```

Implementation: JAF-25 (Week 2)

10. Event-Driven Architecture

10.1 Event Bus Pattern

Singleton Design:

```

# framework/event_bus.py (JAF-28)
from typing import Callable, List
from framework.types import AgentAction
import asyncio

class EventBus:
    """Singleton event bus for broadcasting AgentAction events."""

    _instance = None
    _subscribers: List[Callable[[AgentAction], None]] = []

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def emit(self, action: AgentAction):
        """Emit event to all subscribers (sync)."""
        for subscriber in self._subscribers:
            try:
                subscriber(action)

```

```

        except Exception as e:
            print(f"EventBus: Subscriber error: {e}")

    async def emit_async(self, action: AgentAction):
        """Emit event to all subscribers (async)."""
        tasks = [asyncio.create_task(sub(action)) for sub in self._subscribers]
        await asyncio.gather(*tasks, return_exceptions=True)

    def subscribe(self, callback: Callable[[AgentAction], None]):
        """Subscribe to events."""
        self._subscribers.append(callback)

    def unsubscribe(self, callback: Callable[[AgentAction], None]):
        """Unsubscribe from events."""
        if callback in self._subscribers:
            self._subscribers.remove(callback)

```

Built-in Subscribers:

Console Logger:

```

# framework/subscribers/console_logger.py
from framework.types import AgentAction

def console_subscriber(action: AgentAction):
    """Pretty-print events to console."""
    status_icon = "✅" if action.status == "complete" else "❌"
    print(f"{status_icon} [{action.agentId}] {action.actionType}: {action.message}")

# Register at startup
EventBus().subscribe(console_subscriber)

```

SSE Broadcaster:

```

# framework/subscribers/sse_broadcaster.py
from fastapi import Request
from sse_starlette.sse import EventSourceResponse

class SSEBroadcaster:
    """Broadcasts events to SSE clients."""

```

```

def __init__(self):
    self.clients: Dict[UUID, asyncio.Queue] = {}

def subscribe_process(self, process_id: UUID) -> asyncio.Queue:
    """Subscribe to events for a specific process."""
    queue = asyncio.Queue()
    self.clients[process_id] = queue
    return queue

def event_subscriber(self, action: AgentAction):
    """EventBus callback - broadcasts to SSE clients."""
    # Send to all subscribed clients
    for process_id, queue in self.clients.items():
        queue.put_nowait(action)

# Register at startup
sse_broadcaster = SSEBroadcaster()
EventBus().subscribe(sse_broadcaster.event_subscriber)

# FastAPI endpoint
@app.get("/api/processes/{process_id}/stream")
async def stream_events(process_id: UUID):
    queue = sse_broadcaster.subscribe_process(process_id)

    async def event_generator():
        try:
            while True:
                action = await queue.get()
                yield {"data": action.json()}

                # Stop streaming when process completes
                if action.actionType == "process_complete":
                    break
        except asyncio.CancelledError:
            pass

    return EventSourceResponse(event_generator())

```

Implementation: JAF-28 (Week 2), JAF-30 (Week 3), JAF-14/15 (SSE - Week 3, optional)

11. Error Handling & Recovery

11.1 Error Hierarchy

```
# framework/exceptions.py
class JAFException(Exception):
    """Base exception for all JAF errors."""
    pass

class ActorNotFoundError(JAFException):
    """Actor not found in registry."""
    pass

class InvalidTransitionError(JAFException):
    """Invalid state transition attempted."""
    pass

class ToolExecutionError(JAFException):
    """Tool execution failed."""
    def __init__(self, tool_name: str, error: str):
        self.tool_name = tool_name
        self.error = error
        super().__init__(f"Tool {tool_name} failed: {error}")

class PlanningError(JAFException):
    """Plan generation failed."""
    pass

class EngineTimeoutError(JAFException):
    """Engine execution timeout."""
    pass
```

11.2 Retry Strategy

Tool-Level Retry:

```

# In EngineAbstraction.execute_step()
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=2, max=10),
    reraise=True
)
async def execute_tool_with_retry(tool: Callable, params: Dict) -> Dict:
    """Execute tool with automatic retry on transient errors."""
    try:
        return tool(**params)
    except (TimeoutError, ConnectionError) as e:
        # Transient errors - retry
        raise
    except ValueError as e:
        # Invalid input - don't retry
        raise ToolExecutionError(tool.__name__, str(e))

```

Step-Level Retry:

```

# In Orchestrator loop
result = await self.engine.execute_step(step, context)

if result.status == "error":
    # Evaluate: Should we retry?
    update = await self.engine.evaluate_progress(plan, result)

    if update.action == "retry":
        # Retry same step
        step.status = "pending"
        continue

    elif update.action == "abort":
        # Fail entire process
        await self.state_machine.transition(process_id, "FAILED")
        return {"error": update.reason}

```

11.3 Crash Recovery

Save State After Every Step:

```
# In Orchestrator loop
for step in plan.steps:
    # Execute
    result = await self.engine.execute_step(step, context)

    # Update context
    context[step.id] = result.output

    # ⚠ CRITICAL: Save state BEFORE continuing
    await self.repo.update_context(process_id, context)
    await self.repo.update_plan(process_id, plan) # Mark step as complete

# If crash happens here, we can resume from DB
```

Resume After Crash:

```
# On server restart
async def resume_incomplete_processes():
    """Resume all processes that were executing when server crashed."""

    # Find all non-terminal processes
    incomplete = await repo.find_by_status(["PLANNING", "EXECUTING", "EVALUATING"])

    for process in incomplete:
        process_id = process["id"]
        plan = Plan.parse_raw(process["current_plan"])
        context = json.loads(process["context"])

        # Resume from next pending step
        await orchestrator.resume(process_id, plan, context)
```

Implementation: JAF-25 (Week 2), Demo in Week 3

12. Testing Strategy

12.1 Unit Tests

Actor Tests:

```
# tests/unit/test_provisionning_actor.py
import pytest
from agents.provisionning_agent import ProvisionningAgent

def test_actor_configure():
    """Test actor configuration."""
    actor = ProvisionningAgent()
    actor.configure()

    assert actor.name == "ProvisionningActor"
    assert len(actor.tools) == 7
    assert actor.llm_config.model == "gpt-4o-mini"

def test_check_eligibility():
    """Test check_eligibility tool."""
    actor = ProvisionningAgent()
    result = actor.check_eligibility("C123456")

    assert "eligible" in result
    assert result["eligible"] in [True, False]
```

Engine Tests:

```
# tests/unit/test_crewai_adapter.py
import pytest
from framework.engine.crewai_adapter import CrewAIAdapter
from framework.types import PlanStep

@pytest.mark.asyncio
async def test_execute_step():
    """Test single step execution."""
    engine = CrewAIAdapter()

    step = PlanStep(
        actor_name="ProvisionningActor",
        tool_name="check_eligibility",
        parameters={"cuid": "C123456"}
```

```
)
```

```
result = await engine.execute_step(step, context={})
```

```
assert result.status == "success"
```

```
assert "eligible" in result.output
```

12.2 Integration Tests

End-to-End Flow:

```
# tests/integration/test_end_to_end.py (JAF-37)
```

```
import pytest
```

```
from framework.orchestrator import Orchestrator
```

```
@pytest.mark.asyncio
```

```
async def test_agentic_flow_complete():
```

```
    """Test full agentic flow: input → plan → execute → output."""
```

```
    orchestrator = Orchestrator()
```

```
    result = await orchestrator.run(
```

```
        goal="Provision PC for Alice (cuid: C123456)",
```

```
        mode="agentic"
```

```
    )
```

```
    assert result["status"] == "COMPLETED"
```

```
    assert "process_id" in result
```

```
    assert "result" in result
```

```
    assert result["result"]["model"] == "Dell Latitude 7420"
```

```
@pytest.mark.asyncio
```

```
async def test_multi_agent_flow_complete():
```

```
    """Test full multi-agent flow with delegation."""
```

```
    orchestrator = Orchestrator()
```

```
    result = await orchestrator.run(
```

```
        goal="Onboard Alice: identity + PC + email",
```

```
        mode="multi-agent"
```

```
)
```

```
assert result["status"] == "COMPLETED"
assert "identity" in result["result"]
assert "equipment" in result["result"]
assert "email" in result["result"]
```

```
@pytest.mark.asyncio
```

```
async def test_crash_recovery():
    """Test process resumes after crash."""

    orchestrator = Orchestrator()

    # Start process
    process_id = await orchestrator.start("Provision PC for Alice")

    # Simulate crash (kill orchestrator mid-execution)
    # ... process is in DB with status = "EXECUTING", step 2 of 4 complete

    # Restart orchestrator
    new_orchestrator = Orchestrator()

    # Resume
    result = await new_orchestrator.resume(process_id)

    # Should complete steps 3 and 4
    assert result["status"] == "COMPLETED"
    assert len(result["completed_steps"]) == 4
```

Implementation: JAF-36, JAF-37 (Week 4)

12.3 Mock LLM Service

Purpose: Avoid LLM costs during testing, ensure deterministic results.

Implementation:

```
# tests/mocks/mock_llm.py (JAF-21)
from typing import Dict, List
```

```

class MockChatOpenAI:
    """Mock LLM that returns pre-configured responses."""

    def __init__(self, responses: List[Dict]):
        self.responses = responses
        self.call_count = 0

    async def ainvoke(self, messages: List, **kwargs):
        """Mock async invoke."""
        response = self.responses[self.call_count % len(self.responses)]
        self.call_count += 1
        return response

# Usage in tests
mock_llm = MockChatOpenAI(responses=[
    {
        "content": "I will check eligibility using the check_eligibility tool",
        "tool_calls": [{
            "name": "check_eligibility",
            "arguments": {"cuid": "C123456"}
        }]
    },
    {
        "content": "Eligibility confirmed. I will order the PC.",
        "tool_calls": [{
            "name": "order_product",
            "arguments": {"cuid": "C123456", "model": "Dell Latitude"}
        }]
    }
])

# Inject into engine
engine = CrewAIAdapter(llm=mock_llm)

```

Fixtures for All Agents:

```

# tests/fixtures/provisioning_responses.json
{
    "check_eligibility": {
        "eligible": true,
        "reasons": []
    },

```

```
"select_model": {  
  "model": "Dell Latitude 7420",  
  "generation": "N"  
},  
"order_product": {  
  "order_id": "12345",  
  "status": "confirmed"  
}  
}
```

Implementation: JAF-21 (Week 3)

13. Deployment Architecture

13.1 Local Development

Docker Compose:

```
# docker-compose.dev.yml (JAF-23)  
version: '3.8'  
  
services:  
  postgres:  
    image: postgres:15  
    environment:  
      POSTGRES_DB: jaf_dev  
      POSTGRES_USER: jaf  
      POSTGRES_PASSWORD: jaf123  
    ports:  
      - "5432:5432"  
    volumes:  
      - ./migrations:/docker-entrypoint-initdb.d  
  
  redis:  
    image: redis:7-alpine  
    ports:  
      - "6379:6379"
```

```

api:
  build:
    context: .
    dockerfile: Dockerfile.dev
  volumes:
    - ./app
  environment:
    DATABASE_URL: postgresql+asyncpg://jaf:jaf123@postgres/jaf_dev
    REDIS_URL: redis://redis:6379
    OPENAI_API_KEY: ${OPENAI_API_KEY}
  ports:
    - "8000:8000"
  depends_on:
    - postgres
    - redis
  command: uvicorn main:app --host 0.0.0.0 --reload

frontend:
  image: node:18
  working_dir: /app
  volumes:
    - ./frontend-v0:/app
  ports:
    - "3000:3000"
  command: npm run dev

grafana:
  image: grafana/grafana:latest
  ports:
    - "3001:3000"
  volumes:
    - ./grafana/dashboards:/etc/grafana/provisioning/dashboards

```

One-Command Setup:

```
docker-compose -f docker-compose.dev.yml up
```

Implementation: JAF-23 (Week 4)

13.2 Production Deployment

Dockerfile (Multi-Stage):

```
# Dockerfile (JAF-24)
# Stage 1: Builder
FROM python:3.11-slim as builder

WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

# Stage 2: Runtime
FROM python:3.11-slim

WORKDIR /app

# Copy dependencies from builder
COPY --from=builder /root/.local /root/.local

# Copy application code
COPY . .

# Set environment
ENV PATH=/root/.local/bin:$PATH
ENV PYTHONUNBUFFERED=1

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1

# Run as non-root
RUN useradd -m -u 1000 jaf && chown -R jaf:jaf /app
USER jaf

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Image Size Target: <200MB

Implementation: JAF-24 (Week 4)

14. File Structure (Target State)

```
jems-ai-framework/  
├─ framework/  
|   ├── __init__.py  
|   ├── types.py           # All Pydantic models (JAF-4, JAF-16, JAF-29)  
|   ├── exceptions.py      # Custom exceptions  
|   |  
|   ├── actor/  
|   |   ├── __init__.py  
|   |   ├── base_actor.py  # Actor ABC (JAF-14, JAF-15)  
|   |   └─ registry.py     # ActorRegistry (JAF-17, JAF-18, JAF-19)  
|   |  
|   ├── engine/  
|   |   ├── __init__.py  
|   |   ├── engine_abstraction.py # EngineAbstraction ABC (JAF-2, JAF-3)  
|   |   ├── crewai_adapter.py    # CrewAI wrapper (JAF-5, JAF-6, JAF-7, JAF-8)  
|   |   ├── native_engine.py    # Native implementation (JAF-15, optional)  
|   |   └─ factory.py           # EngineFactory (JAF-16)  
|   |  
|   ├── persistence/  
|   |   ├── __init__.py  
|   |   ├── database.py        # Connection management  
|   |   ├── models.py          # SQLAlchemy models  
|   |   └─ process_repo.py     # Repository (JAF-23, JAF-24, JAF-25)  
|   |  
|   ├── event_bus.py          # EventBus singleton (JAF-27, JAF-28)  
|   ├── state_machine.py      # ProcessStateMachine (JAF-19)  
|   └─ orchestrator.py        # Main orchestration loop (Future)  
|  
├─ agents/  
|   ├── __init__.py  
|   ├── provisionning_agent.py # Migrated to Actor (JAF-20, JAF-21)  
|   ├── piramid_agent.py      # To be migrated (Week 3)  
|   ├── habilitation_agent.py # To be migrated (Week 3)  
|   ├── geai_agent.py         # To be migrated (Week 3)  
|   └─ annuaire_agent.py      # To be migrated (Week 3)  
|  
└─ examples/
```

```

|   ├── agentic_flow_example.py      # Demo Outcome #1 (JAF-32, JAF-33)
|   ├── multi_agent_flow_example.py # Demo Outcome #2 (JAF-34, JAF-35)
|   └── traditional_automation_example.py # Demo Outcome #3 (JAF-40, optional)
|
└── tests/
    ├── unit/
    |   ├── test_actors.py
    |   ├── test_engine.py
    |   └── test_types.py
    ├── integration/
    |   └── test_end_to_end.py      # Full E2E test (JAF-36, JAF-37)
    ├── mocks/
    |   └── mock_llm.py            # Mock LLM service (JAF-21)
    └── fixtures/
        └── *.json                 # Test data
|
└── migrations/
    └── 001_create_processes.sql    # DB migration (JAF-24)
|
└── docs/
    ├── JAF_ARCHITECTURE_GUIDE.md # This file
    ├── JAF_BACKLOG_FINAL.md      # Sprint backlog
    ├── FRAMEWORK_SPEC.md         # Technical spec
    └── AI_CONTEXT_QUICK.md       # AI handoff doc
|
└── main.py                       # FastAPI entry point
└── requirements.txt
└── docker-compose.dev.yml        # Dev environment (JAF-23)
└── Dockerfile                    # Production image (JAF-24)

```

15. Implementation Timeline (4-Week Roadmap)

Week 1: Foundation (Files to Create)

Priority 1 (MUST COMPLETE):

-  framework/types.py (JAF-4, JAF-16, JAF-29)
-  framework/engine/engine_abstraction.py (JAF-2, JAF-3)

-  framework/actor/base_actor.py (JAF-14, JAF-15)
-  framework/actor/registry.py (JAF-17, JAF-18, JAF-19)
-  migrations/001_create_processes.sql (JAF-24)

Priority 2 (START):

-  framework/engine/crewai_adapter.py (JAF-5) - Prototype only
-  agents/provisionning_agent.py (JAF-20, JAF-21) - Migration started

Deliverable: Architecture validated, core types defined, 1 agent migration 20% complete

Week 2: Execution (Files to Create)

Priority 1 (MUST COMPLETE):

-  framework/engine/crewai_adapter.py (JAF-5, JAF-6, JAF-7, JAF-8) - Full implementation
-  agents/provisionning_agent.py (JAF-20, JAF-21) - Migration complete
-  framework/persistence/process_repo.py (JAF-25)
-  framework/event_bus.py (JAF-28) - Basic version
-  examples/agentic_flow_example.py (JAF-32, JAF-33) - Started

Priority 2 (START):

-  framework/engine/factory.py (JAF-16)
-  examples/multi_agent_flow_example.py (JAF-34) - Planning

Deliverable: Agentic flow working end-to-end, CrewAI adapter functional

Week 3: Integration (Files to Create)

Priority 1 (MUST COMPLETE):

-  examples/multi_agent_flow_example.py (JAF-34, JAF-35) - Complete
-  framework/state_machine.py (JAF-19)

-  SSE endpoint in `main.py` (JAF-14, JAF-15) - If time permits

Priority 2 (MIGRATE):

-  `agents/piramid_agent.py` - Migrate to Actor
-  `agents/habilitation_agent.py` - Migrate to Actor
-  `agents/geai_agent.py` - Migrate to Actor
-  `agents/annuaire_agent.py` - Migrate to Actor

Deliverable: Multi-agent flow working, all agents migrated, crash recovery demo ready

Week 4: Production (Files to Create)

Priority 1 (MUST COMPLETE):

-  `tests/integration/test_end_to_end.py` (JAF-36, JAF-37)
-  `tests/mocks/mock_llm.py` (JAF-21)
-  `docker-compose.dev.yml` (JAF-23)
-  `Dockerfile` (JAF-24)

Priority 2 (OPTIONAL):

-  `framework/engine/native_engine.py` (JAF-15) - If time
-  `examples/traditional_automation_example.py` (JAF-40) - If time
-  Grafana dashboards (JAF-22) - If time

Deliverable: Client demo ready, all tests passing, production deployment working

16. Code Quality Standards

16.1 Type Safety (MANDATORY)

All code must:

-  Have type hints for all function parameters and return values
-  Pass `mypy --strict` with no errors
-  Use Pydantic models for data validation

Example:

```
#  BAD: No type hints
def check_eligibility(cuid):
    return {"eligible": True}

#  GOOD: Type hints + Pydantic
from typing import Dict

def check_eligibility(self, cuid: str) -> Dict[str, Any]:
    """Check if user is eligible for equipment.

    Args:
        cuid: User corporate ID (format: C + 6 digits)

    Returns:
        Dict with keys: eligible (bool), reasons (List[str])
    """
    result = self._api_call(cuid)
    return {"eligible": result.get("eligible", False), "reasons": []}
```

16.2 Error Handling (MANDATORY)

All tools must:

-  Return structured dicts (never throw unhandled exceptions)
-  Include error key if operation fails
-  Log errors with structured logger

Pattern:

```
def my_tool(self, param: str) -> Dict[str, Any]:
    """Tool description."""
    try:
        result = self._external_api_call(param)
```

```
        return {"success": True, "data": result}

    except TimeoutError as e:
        # Transient error - engine can retry
        return {"success": False, "error": "timeout", "retryable": True}

    except ValueError as e:
        # Invalid input - don't retry
        return {"success": False, "error": str(e), "retryable": False}

    except Exception as e:
        # Unknown error - log and return
        self.logger.error("Unexpected error in my_tool", error=str(e))
        return {"success": False, "error": "internal_error", "retryable": False}
```

16.3 Logging (MANDATORY)

Use Structured Logging:

```
# Import
from utils.observability import StructuredLogger

# Initialize
logger = StructuredLogger(__name__)

# Log with context
logger.info("Checking eligibility", cuid=cuid, actor="ProvisionningActor")
logger.error("API call failed", endpoint="/eligibility", status_code=500, error=str
```

DON'T use print() in production code (use logger instead).

16.4 Documentation (MANDATORY)

All classes and functions must have docstrings:

```
def my_tool(self, cuid: str, model: str) -> Dict[str, Any]:
    """Order hardware for employee.

    This tool calls the SIGALE API to place an order for the specified
    hardware model. It handles stock checks and delivery tracking.

    Args:
        cuid: Employee corporate ID (format: C + 6 digits)
        model: Hardware model (e.g., "Dell Latitude 7420")

    Returns:
        Dict with keys:
            - order_id (str): SIGALE order number
            - status (str): Order status (confirmed/pending/failed)
            - delivery_date (str): Expected delivery (ISO format)
            - error (str, optional): Error message if failed

    Raises:
        ValueError: If cuid format is invalid
        ConnectionError: If SIGALE API is unreachable

    Example:
        >>> actor.my_tool("C123456", "Dell Latitude 7420")
        {"order_id": "12345", "status": "confirmed", "delivery_date": "2024-12-05"}
        """
    # Implementation...
```

17. Architecture Decision Records (ADRs)

ADR-001: Why Actor Model over Standard OOP?

Status:  Accepted

Date: 30 Nov 2024

Decision Maker: Alex (Team Lead)

Context:

- Current system uses standard OOP (agents as objects)
- Agents share global state (config, memory)
- Hard to scale, test, deploy independently

Decision:

Adopt Actor Model (Erlang/Akka pattern) for framework.

Rationale:

1. **Share Nothing:** Eliminates race conditions, deadlocks
2. **Message Passing:** Enables location transparency (local → remote)
3. **Fault Tolerance:** Supervision hierarchies (let it crash)
4. **Proven:** 50 years of distributed systems theory (Hewitt 1973)

Consequences:

-  Actors can be deployed as microservices later
 -  Testing is easier (isolated units)
 -  Learning curve for developers (new paradigm)
-

ADR-002: Why Engine Abstraction over Direct CrewAI?

Status:  Accepted

Date: 30 Nov 2024

Decision Maker: Alex (Team Lead)

Context:

- Currently hardcoded to CrewAI
- Can't switch to LangGraph, Vertex AI, etc.
- CrewAI might sunset or raise prices

Decision:

Abstract execution behind `EngineAbstraction` interface.

Rationale:

1. **Vendor Independence:** Can swap backends without changing actor code
2. **Cost Optimization:** Can use cheap engine for simple tasks
3. **Future-Proof:** New execution frameworks emerge constantly

Consequences:

-  Actors never import CrewAI directly
 -  Can A/B test engines (performance, cost)
 -  Extra abstraction layer (slight complexity increase)
-

ADR-003: Why Postgres over In-Memory State?

Status:  Accepted

Date: 30 Nov 2024

Decision Maker: Alex (Team Lead)

Context:

- Current system keeps state in memory only
- Crash = lost workflow state
- No audit trail for compliance

Decision:

Persist process state to Postgres after every step.

Rationale:

1. **Crash Recovery:** Can resume exactly where process crashed
2. **Human-in-the-Loop:** Can pause/resume workflows (approval flows)
3. **Audit Trail:** Regulatory compliance (who did what when)
4. **Long-Running Workflows:** Onboarding takes days/weeks

Consequences:

-  99.9% reliability (zero data loss)
-  Supports enterprise workflows
-  Slight performance overhead (~50ms per step to save)

ADR-004: Why 3 Execution Modes?

Status:  Accepted

Date: 30 Nov 2024

Decision Maker: Alex (Team Lead) + Alexey (Product Owner)

Context:

- Different workflows have different requirements
- Cost vs intelligence trade-off
- Compliance requires deterministic flows

Decision:

Support 3 modes: Agentic, Multi-Agent, Traditional.

Rationale:

1. **Agentic (80% of tasks):** Fast, cheap, sufficient for simple workflows
2. **Multi-Agent (15% of tasks):** Complex, intelligent, worth extra cost
3. **Traditional (5% of tasks):** Compliance-critical, zero AI risk

Consequences:

-  Cost optimization (50% savings overall)
-  Flexibility (choose right tool for job)
-  Increased complexity (3 code paths to maintain)

18. Security & Compliance

18.1 API Authentication (Future)

JWT-Based Auth:

```

# Future implementation (Week 5+)
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

security = HTTPBearer()

async def verify_token(token: str = Depends(security)):
    """Verify JWT token."""
    # Decode and validate token
    pass

@app.post("/api/processes", dependencies=[Depends(verify_token)])
async def create_process(goal: str):
    # Only authenticated users can create processes
    pass

```

18.2 Secrets Management

Environment Variables:

```

# config/settings.py
from pydantic import BaseSettings

class Settings(BaseSettings):
    database_url: str
    openai_api_key: str
    redis_url: str

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"

settings = Settings()

```

Never commit secrets to Git:

```

# .env (gitignored)
DATABASE_URL=postgresql+asyncpg://user:pass@localhost/jaf

```

OPENAI_API_KEY=sk- ...
REDIS_URL=redis://localhost:6379

19. Performance Targets

19.1 Latency Targets

Component	Target	Measurement
Engine.generate_plan()	<1s	LLM planning call
Engine.execute_step()	<500ms	Single tool execution
Persistence (save state)	<50ms	Postgres write
Event emission	<10ms	Event bus broadcast
Full Agentic Flow	<2s	End-to-end (5 steps)
Full Multi-Agent Flow	<5s	End-to-end (5 steps, 3 actors)

19.2 Cost Targets

Mode	Target Cost	Measurement
Agentic Flow	<\$0.05	gpt-4o-mini for routing
Multi-Agent Flow	<\$0.12	Multiple gpt-4o-mini calls
Traditional Automation	\$0.00	No LLM calls

Current System (Baseline): ~\$0.15 per workflow
Target Savings: 50% (use Agentic for 80% of workflows)

20. Monitoring & Observability (Week 4, Optional)

20.1 Metrics to Track

Process Metrics:

- Total processes created (counter)
- Process success rate (gauge)
- Average process duration (histogram)
- Processes by status (gauge per status)

Actor Metrics:

- Tool execution count per actor (counter)
- Tool success rate per actor (gauge)
- Tool execution time per actor (histogram)

Cost Metrics:

- Total LLM token usage (counter)
- Cost per process (histogram)
- Cost by actor (counter)

Implementation:

```
# framework/metrics.py (JAF-22)
from prometheus_client import Counter, Histogram, Gauge

process_counter = Counter('jaf_processes_total', 'Total processes created')
process_duration = Histogram('jaf_process_duration_seconds', 'Process execution time')
tool_executions = Counter('jaf_tool_executions', 'Tool executions', ['actor', 'tool'])
llm_cost = Counter('jaf_llm_cost_usd', 'LLM cost in USD')

# In engine
@trace_operation("execute_step")
async def execute_step(self, step: PlanStep, context: dict) -> StepResult:
    with process_duration.time():
        result = await self._execute(step, context)
        tool_executions.labels(actor=step.actor_name, tool=step.tool_name).inc()
```

```
llm_cost.inc(result.cost)
```

21. Summary & Quick Reference

For Developers Starting Tasks:

Before coding:

1. Read this architecture guide
2. Read your specific JAF-X story in `JAF_BACKLOG_FINAL.md`
3. Check dependencies (what must be complete first?)
4. Review acceptance criteria
5. Ask Alex (Team Lead) if anything unclear

While coding:

1. Follow type safety rules (mypy strict)
2. Write unit tests as you go (>70% coverage)
3. Use structured logging (no `print()`)
4. Document with docstrings
5. Commit frequently to feature branch

Before merging:

1. All tests pass (unit + integration)
2. Code reviewed by supervisor or Alex
3. Documentation updated
4. Demo prepared (can show in Friday demo)

Key Files Quick Reference:

File	Purpose	JAF Story	Week
framework/types.py	All Pydantic models	JAF-4, JAF-16, JAF-29	Week 1
framework/engine/engine_abstraction.py	Engine interface	JAF-2, JAF-3	Week 1
framework/actor/base_actor.py	Actor base class	JAF-14, JAF-15	Week 1
framework/engine/crewai_adapter.py	CrewAI wrapper	JAF-5	Week 2
agents/provisionning_agent.py	Pilot migration	JAF-20, JAF-21	Week 1-2
examples/agentic_flow_example.py	Demo Outcome #1	JAF-32, JAF-33	Week 2-4
examples/multi_agent_flow_example.py	Demo Outcome #2	JAF-34, JAF-35	Week 3-4
tests/integration/test_end_to_end.py	E2E validation	JAF-36, JAF-37	Week 4

END OF ARCHITECTURE GUIDE

Version: 1.0

Author: Alex (Team Lead / Senior Architect)

Approved By: Alexey (Product Owner) - Pending

Status:  Ready for Implementation

Next Step: Week 1 Monday kickoff, start coding JAF-3, JAF-15, JAF-24